



GUÍA DE LABORATORIO N° 01

Ciclo de Vida de Proyectos de Machine Learning

1. Objetivos

Al finalizar esta sesión de laboratorio, el estudiante será capaz de:

Objetivo general: Implementar el ciclo de vida completo de un proyecto de Machine Learning — desde la recolección de datos hasta la serialización del modelo — utilizando herramientas del ecosistema Scikit-learn sobre un caso clínico real.

Objetivos específicos:

- Cargar y explorar un dataset público (Heart Disease UCI) identificando sus características, distribuciones y correlaciones.
- Preparar los datos para el modelado mediante limpieza, split estratificado y estandarización.
- Entrenar y comparar al menos tres modelos de clasificación utilizando pipelines y validación cruzada.
- Evaluar el rendimiento del mejor modelo con métricas de clasificación, matriz de confusión y curva ROC.
- Serializar el modelo entrenado con joblib e implementar una función de inferencia que simule un endpoint de despliegue.

2. Desarrollo del Laboratorio

El laboratorio se estructura siguiendo las seis etapas del ciclo de vida de un proyecto de Machine Learning. Cada etapa se desarrolla en el notebook de Google Colab adjunto.

2.1. Configuración del entorno

Abra el notebook en Google Colab utilizando el enlace proporcionado en el repositorio del curso. El notebook ya incluye las importaciones necesarias: NumPy, Pandas, Matplotlib, Seaborn y los módulos de Scikit-learn. Ejecute la primera celda para verificar que el entorno está correctamente configurado.

2.2. Etapa 1 — Recolección de datos

Se carga el dataset Heart Disease (Cleveland) desde una URL pública del repositorio UCI Machine Learning. Este dataset contiene 303 registros de pacientes con 13 variables clínicas predictoras y una variable objetivo binaria (presencia/ausencia de enfermedad cardíaca). El notebook incluye un mecanismo de respaldo que, si falla la URL principal, descarga los datos desde OpenML.

2.3. Etapa 2 — Exploración y preparación de datos

En esta etapa se realiza un análisis exploratorio completo:

- a) Inspección general del dataset con `.info()` y `.describe()` para identificar tipos de datos y estadísticas descriptivas.
- b) Verificación de valores nulos y registros duplicados, aplicando limpieza cuando corresponda.
- c) Visualización de la distribución de la variable objetivo (balance de clases).
- d) Distribución de las variables numéricas clave segmentadas por la clase objetivo.
- e) Matriz de correlación para identificar relaciones lineales entre variables.

2.4. Etapa 3 — Preparación para el modelado

Se separan las features (X) del target (y) y se realiza un split estratificado 80/20 con semilla fija (`random_state=42`) para garantizar reproducibilidad. El estratificado asegura que ambas particiones mantengan la misma proporción de clases.

2.5. Etapa 4 — Entrenamiento de modelos

Se definen tres pipelines con los siguientes modelos:

- a) Regresión Logística: modelo lineal con estandarización previa (`StandardScaler`). Sirve como baseline robusto para clasificación binaria.
- b) Árbol de Decisión: modelo no lineal con profundidad máxima de 5. No requiere estandarización.
- c) Random Forest: ensamble de 200 árboles con profundidad máxima de 7. Generalmente ofrece mejor rendimiento que un árbol individual.

Cada pipeline se evalúa con validación cruzada de 5 folds sobre el conjunto de entrenamiento. Se comparan visualmente los resultados mediante boxplots.

2.6. Etapa 5 — Evaluación del mejor modelo

El modelo con mayor accuracy promedio en validación cruzada se entrena sobre el conjunto completo de entrenamiento y se evalúa sobre el conjunto de test. Se generan las siguientes métricas y visualizaciones:

- a) Classification report: precision, recall y F1-score por clase.
- b) Matriz de confusión: verdaderos positivos/negativos y falsos positivos/negativos.
- c) Curva ROC y AUC: capacidad de discriminación del modelo en diferentes umbrales.

2.7. Etapa 6 — Serialización y despliegue conceptual

Se serializa el pipeline completo (preprocesamiento + modelo) con `joblib`, permitiendo su reutilización sin necesidad de reentrenar. Se implementa una función de inferencia `predict_heart_disease()` que simula cómo operaría un endpoint en producción: recibe los datos de un paciente como diccionario y retorna la predicción con sus probabilidades.

3. Ejercicios Propuestos

Estos ejercicios son de resolución individual. Deben entregarse en un notebook (.ipynb) adicional vía el repositorio del curso.

Ejercicio 1 — Optimización de hiperparámetros

Utilice `GridSearchCV` o `RandomizedSearchCV` para optimizar los hiperparámetros del `RandomForestClassifier` (por ejemplo: `n_estimators`, `max_depth`, `min_samples_split`). Compare el `accuracy` antes y después del tuning, reportando los mejores hiperparámetros encontrados y la mejora obtenida.

Ejercicio 2 — Replicación del ciclo de vida con otro dataset

Seleccione uno de los siguientes datasets y replique el ciclo de vida completo desarrollado en la sesión:

- a) Breast Cancer Wisconsin: `sklearn.datasets.load_breast_cancer` (clasificación binaria).
- b) Diabetes: `sklearn.datasets.load_diabetes` (regresión — adapte las métricas de evaluación).

Documente cada etapa y las diferencias encontradas respecto al caso de Heart Disease.

Ejercicio 3 — Registro de experimentos con MLflow

Investigue la librería `MLflow` e implemente un registro básico de los experimentos realizados. Para cada modelo entrenado, registre al menos: nombre del modelo, hiperparámetros, `accuracy`, `precision` y `recall`. Capture una imagen del dashboard de `MLflow` mostrando los experimentos registrados.

Entregables: (1) Notebook guiado ejecutado (.ipynb), (2) Notebook de ejercicios (.ipynb). Ambos deben subirse al repositorio del curso antes de la siguiente sesión.